

ACME 1000 Roadster

test123

Table of Contents

Code Topic	3
test 1	3
test 2	3
test 3	3
Rotate image	4
gff	7
ACME 1000 Roadster	8
SPF+++++	8
test 1	8
Rotate image	8
gff	11
ad	12
Test drag	12
Link test 123	13
accoridon	14
Sample publication	15
test paste	15
Overlaps on the bottom	15
hhfjkd	15
32783672gfhj	16
hkfdhsfjkdhsjkfds	16
hkfdhsfjkdhsjkfds	17
.....	18
test topic translate	18

Code Topic

Here is a link with autogenerated title here [Test](#)

Here is a link with the title set [here](#)

[CKEditor test](#)

test 1

ddsdas

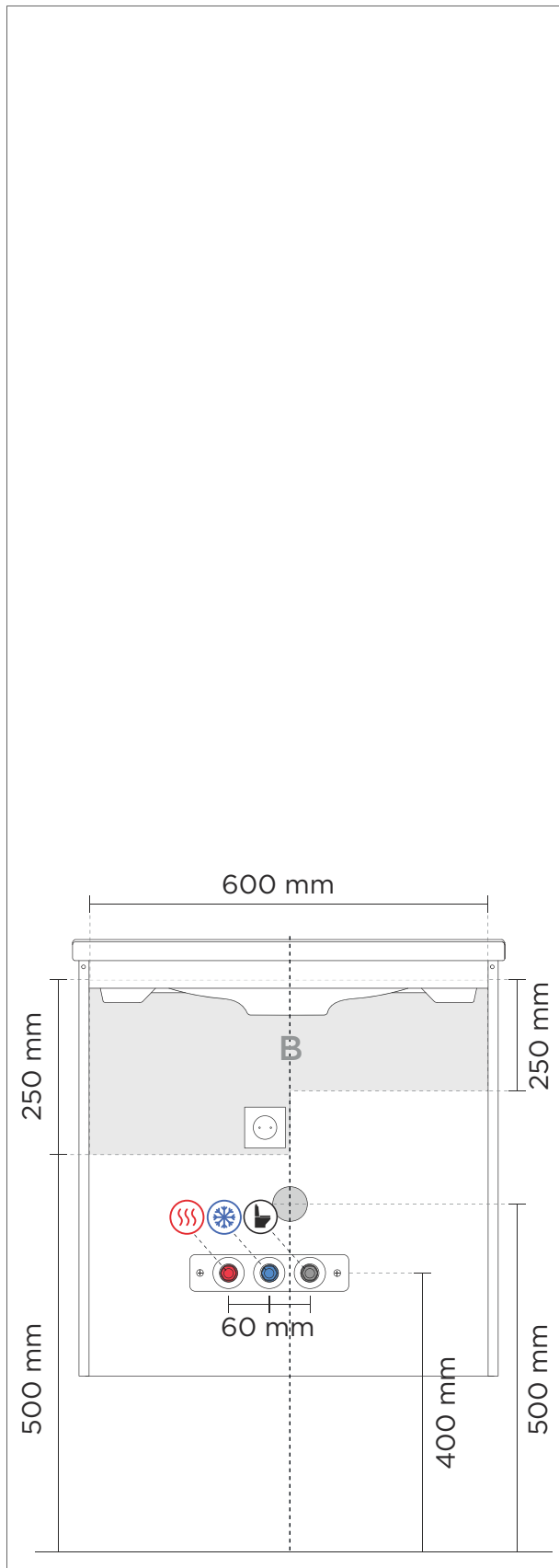
test 2

ddsdas

test 3

ddsdas

Rotate image



```

$(document).ready(function () {
    // Your existing code
    var icon1 = "https://documents.o9solutions.com/wp-content/homeicons/referenceproduct.png";
    var icon2 = "https://documents.o9solutions.com/wp-content/homeicons/uinavigation.png";
    var icon3 = "https://documents.o9solutions.com/wp-content/homeicons/dataintegration.png";
    var icon4 = "https://documents.o9solutions.com/wp-content/homeicons/supplyplanning.png";
    var icon5 = "https://documents.o9solutions.com/wp-content/homeicons/demandplanning.png";
    var icon6 = "https://documents.o9solutions.com/wp-content/homeicons/orderpromising.png";
    var icon7 = "https://documents.o9solutions.com/wp-content/homeicons/knowledgehub.png";
    var icon8 = "https://documents.o9solutions.com/wp-content/homeicons/sustainability.png";
    var icon9 = "https://documents.o9solutions.com/wp-content/homeicons/productionschedule.png";
    var icon10 = "https://documents.o9solutions.com/wp-content/homeicons/nextgenprocurement.png";
    var icon11 = "https://documents.o9solutions.com/wp-content/homeicons/retail.png";
    var icon12 = "https://documents.o9solutions.com/wp-content/homeicons/meio.png";
    var icon13 = "https://documents.o9solutions.com/wp-content/homeicons/oio.png";
    var icon14 = "https://documents.o9solutions.com/wp-content/homeicons/aisegmentation.png";
    var icon15 = "https://documents.o9solutions.com/wp-content/homeicons/segmentationfour.png";
    var icon16 = "https://documents.o9solutions.com/wp-content/homeicons/rgm.png";

    $(".cat-panel-1").closest("a").prepend("<img src='" + icon1 + "'></img>");
    $(".cat-panel-2").closest("a").prepend("<img src='" + icon2 + "'></img>");
    $(".cat-panel-3").closest("a").prepend("<img src='" + icon3 + "'></img>");
    $(".cat-panel-4").closest("a").prepend("<img src='" + icon4 + "'></img>");
    $(".cat-panel-5").closest("a").prepend("<img src='" + icon5 + "'></img>");
    $(".cat-panel-6").closest("a").prepend("<img src='" + icon6 + "'></img>");
    $(".cat-panel-7").closest("a").prepend("<img src='" + icon7 + "'></img>");
    $(".cat-panel-8").closest("a").prepend("<img src='" + icon8 + "'></img>");
    $(".cat-panel-9").closest("a").prepend("<img src='" + icon9 + "'></img>");
    $(".cat-panel-10").closest("a").prepend("<img src='" + icon10 + "'></img>");
    $(".cat-panel-11").closest("a").prepend("<img src='" + icon11 + "'></img>");
    $(".cat-panel-12").closest("a").prepend("<img src='" + icon12 + "'></img>");
    $(".cat-panel-13").closest("a").prepend("<img src='" + icon13 + "'></img>");
    $(".cat-panel-14").closest("a").prepend("<img src='" + icon14 + "'></img>");
    $(".cat-panel-15").closest("a").prepend("<img src='" + icon15 + "'></img>");
    $(".cat-panel-16").closest("a").prepend("<img src='" + icon16 + "'></img>");

    $(".publication-icon").remove();

    // New code
    setWelcomeText();
    $(document).ajaxComplete(function () {
        setWelcomeText();
    });

    // Find the feedback panel element
    var feedbackElement = $("#email-feedback");

    if (feedbackElement.length > 0) {
        var anchorElement = feedbackElement.find('a');

        if (anchorElement.length > 0) {
            anchorElement.text('Feedback');
        }

        // Move the feedback panel element to the footer

```

```

        moveFeedbackPanelToFooter(feedbackElement);
    }

    // Additional script to move email-feedback div inside the inner div of the footer
    var footer = document.querySelector('.site-footer');
    var innerDiv = footer.querySelector('.inner');
    var emailFeedbackDiv = footer.querySelector('#email-feedback');
    innerDiv.appendChild(emailFeedbackDiv);
});

function setWelcomeText() {
    for (var key in welcomeTextDict) {
        var element = $("header.portal-header[data-portal-language=" + key + "] h1");
        element.text(welcomeTextDict[key]);
    }
}

function moveFeedbackPanelToFooter(feedbackElement) {
    // Find the footer element
    var footer = $(".site-footer");

    // Check if both elements exist
    if (feedbackElement && feedbackElement.length > 0 && footer.length > 0) {
        // Detach the feedback panel from its current position
        feedbackElement.detach();

        // Append the feedback panel to the footer
        footer.append(feedbackElement);
    }
}

var welcomeTextDict = { "en": "Welcome to o9 Documentation!" };

var auth0 = new Auth0({
    domain: 'dev-vjyg2r1w3qj5ovqa.us.auth0.com',
    clientId: 'ImypxJQmZM2wvlav6KarbHL0xu1HhByC',
    callbackOnLocationHash: true,
    callbackURL: window.location.href
});

// Trigger login when a button is clicked
document.getElementById('login-button').addEventListener('click', function () {
    auth0.login();
});

Areaspec error++

{Areaspec error++
"LEM-EmpfaengerId":"guid",
"pmId" : "String",
"Adresse" : {
"Name" : "String",
"Strasse" : "String",
"PLZ" : "String",
"Stadt" : "String",
"Land" : "String"

```

```
}  
"Hersteller" : [  
{  
  "Hersteller" : "String",  
  "Hersteller-Kunde-Nr" : "",  
  "E-Mails" : [ "E-Mail1"]  
}  
]  
}
```

gff
ff

ACME 1000 Roadster

SPF+++++

adding so

fhedfghedsk

hello here is some bad grammer ghj, dont rember ggrgre

Test

testing some css rules

- Item 1
 - Item 1.1
 - Item 2.2
- Item2
- Item 3

code goes heresome test content for a literal layout

testing [glossary](#)

Glossary title

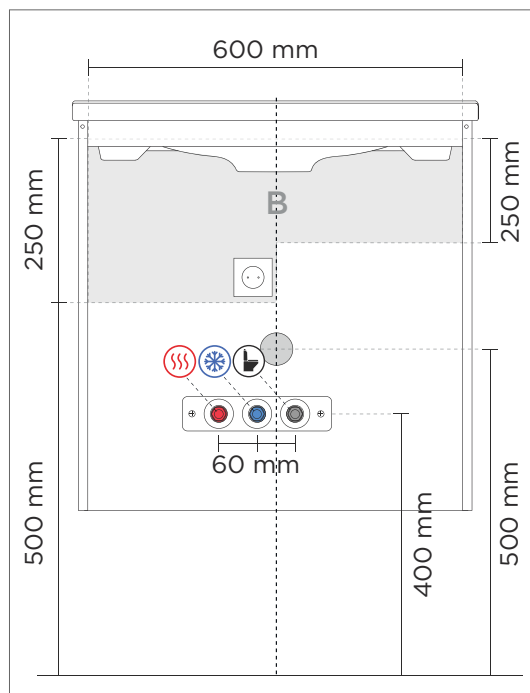
test term ()

test def

test 1

adding some text here

Rotate image



```

$(document).ready(function () {
    // Your existing code
    var icon1 = "https://documents.o9solutions.com/wp-content/homeicons/referenceproduct.png";
    var icon2 = "https://documents.o9solutions.com/wp-content/homeicons/uinavigation.png";
    var icon3 = "https://documents.o9solutions.com/wp-content/homeicons/dataintegration.png";
    var icon4 = "https://documents.o9solutions.com/wp-content/homeicons/supplyplanning.png";
    var icon5 = "https://documents.o9solutions.com/wp-content/homeicons/demandplanning.png";
    var icon6 = "https://documents.o9solutions.com/wp-content/homeicons/orderpromising.png";
    var icon7 = "https://documents.o9solutions.com/wp-content/homeicons/knowledgehub.png";
    var icon8 = "https://documents.o9solutions.com/wp-content/homeicons/sustainability.png";
    var icon9 = "https://documents.o9solutions.com/wp-content/homeicons/productionschedule.png";
    var icon10 = "https://documents.o9solutions.com/wp-content/homeicons/nextgenprocurement.png";
    var icon11 = "https://documents.o9solutions.com/wp-content/homeicons/retail.png";
    var icon12 = "https://documents.o9solutions.com/wp-content/homeicons/meio.png";
    var icon13 = "https://documents.o9solutions.com/wp-content/homeicons/oio.png";
    var icon14 = "https://documents.o9solutions.com/wp-content/homeicons/aisegmentation.png";
    var icon15 = "https://documents.o9solutions.com/wp-content/homeicons/segmentationfour.png";
    var icon16 = "https://documents.o9solutions.com/wp-content/homeicons/rgm.png";

    $(".cat-panel-1").closest("a").prepend("<img src='" + icon1 + "'></img>");
    $(".cat-panel-2").closest("a").prepend("<img src='" + icon2 + "'></img>");
    $(".cat-panel-3").closest("a").prepend("<img src='" + icon3 + "'></img>");
    $(".cat-panel-4").closest("a").prepend("<img src='" + icon4 + "'></img>");
    $(".cat-panel-5").closest("a").prepend("<img src='" + icon5 + "'></img>");
    $(".cat-panel-6").closest("a").prepend("<img src='" + icon6 + "'></img>");
    $(".cat-panel-7").closest("a").prepend("<img src='" + icon7 + "'></img>");
    $(".cat-panel-8").closest("a").prepend("<img src='" + icon8 + "'></img>");
    $(".cat-panel-9").closest("a").prepend("<img src='" + icon9 + "'></img>");
    $(".cat-panel-10").closest("a").prepend("<img src='" + icon10 + "'></img>");
    $(".cat-panel-11").closest("a").prepend("<img src='" + icon11 + "'></img>");
    $(".cat-panel-12").closest("a").prepend("<img src='" + icon12 + "'></img>");
    $(".cat-panel-13").closest("a").prepend("<img src='" + icon13 + "'></img>");
    $(".cat-panel-14").closest("a").prepend("<img src='" + icon14 + "'></img>");
    $(".cat-panel-15").closest("a").prepend("<img src='" + icon15 + "'></img>");
    $(".cat-panel-16").closest("a").prepend("<img src='" + icon16 + "'></img>");

    $(".publication-icon").remove();

    // New code
    setWelcomeText();
    $(document).ajaxComplete(function () {
        setWelcomeText();
    });

    // Find the feedback panel element
    var feedbackElement = $("#email-feedback");

    if (feedbackElement.length > 0) {
        var anchorElement = feedbackElement.find('a');

        if (anchorElement.length > 0) {
            anchorElement.text('Feedback');
        }

        // Move the feedback panel element to the footer

```

```

        moveFeedbackPanelToFooter(feedbackElement);
    }

    // Additional script to move email-feedback div inside the inner div of the footer
    var footer = document.querySelector('.site-footer');
    var innerDiv = footer.querySelector('.inner');
    var emailFeedbackDiv = footer.querySelector('#email-feedback');
    innerDiv.appendChild(emailFeedbackDiv);
});

function setWelcomeText() {
    for (var key in welcomeTextDict) {
        var element = $("header.portal-header[data-portal-language=" + key + "] h1");
        element.text(welcomeTextDict[key]);
    }
}

function moveFeedbackPanelToFooter(feedbackElement) {
    // Find the footer element
    var footer = $(".site-footer");

    // Check if both elements exist
    if (feedbackElement && feedbackElement.length > 0 && footer.length > 0) {
        // Detach the feedback panel from its current position
        feedbackElement.detach();

        // Append the feedback panel to the footer
        footer.append(feedbackElement);
    }
}

var welcomeTextDict = { "en": "Welcome to o9 Documentation!" };

var auth0 = new Auth0({
    domain: 'dev-vjyg2r1w3qj5ovqa.us.auth0.com',
    clientId: 'ImypxJQmZM2wvlav6KarbHL0xu1HhByC',
    callbackOnLocationHash: true,
    callbackURL: window.location.href
});

// Trigger login when a button is clicked
document.getElementById('login-button').addEventListener('click', function () {
    auth0.login();
});

Areaspec error++

{Areaspec error++
"LEM-EmpfaengerId":"guid",
"pmId" : "String",
"Adresse" : {
"Name" : "String",
"Strasse" : "String",
"PLZ" : "String",
"Stadt" : "String",
"Land" : "String"

```

```
}  
"Hersteller" : [  
{  
  "Hersteller" : "String",  
  "Hersteller-Kunde-Nr" : "",  
  "E-Mails" : [ "E-Mail1"]  
}  
]  
}
```

gff
ff

ad

☐ test <http://acme.com/api/>

hello world hello world hello world hello world hello world hello world hel-
lo world hello world hello world hello world hello world hello world hello
world hello world hello world hello world

hello word

Test drag

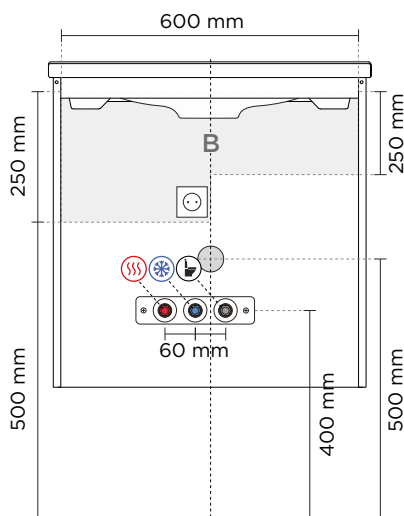
dsa

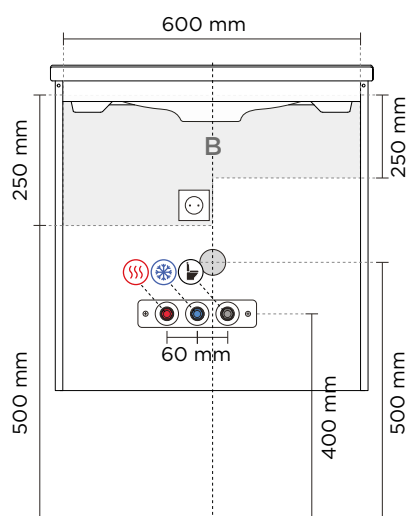
Link test 123

find the link below

transaltion

english version





accoridon

test

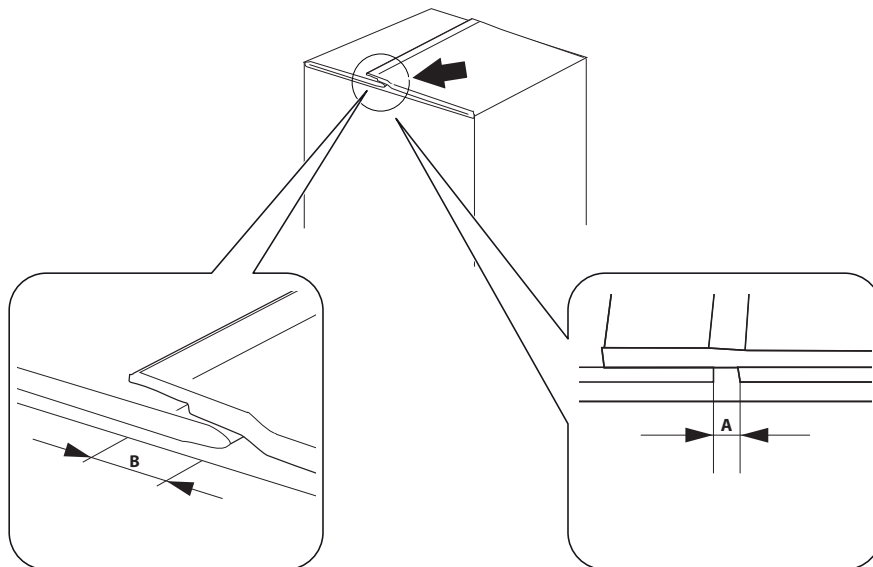
Sample publication

test paste

Overlaps on the bottom

- Check that space (**A**) has the value shown in the table.
- Check that overlap (**B**) has the value shown in the table.**abc**

Format	Space (A)	Overlap (B)
70x70 ≤ 500ml	≤ 1.5 mm	≤ 1.5 mm
	≤ 2 mm	≤ 2 mm
70x70 > 600ml	≤ 1.0 mm	≤ 1.0 mm
	≤ 1.5 mm	≤ 1.5 mm



List filter example

-RS-hhfjkd

4. List item 1
5. List item 2
6. List item 3

An element splitting the list

7. List item 4
8. List item 5phrase 1phrase 2

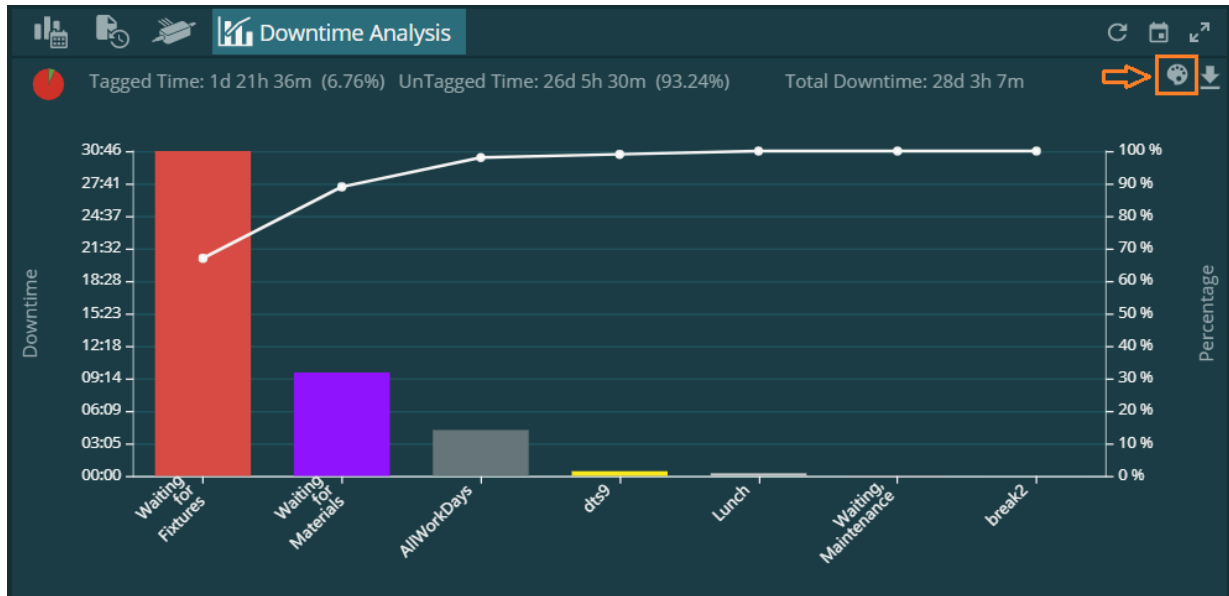


WARNING

fjdshfdkj

Table topic sample

Here is some text with the ~~strikethrough~~ role added.



34343



-RS-hkfdhsfjkdhsjkfds

Table 1. table

Table 2.

Table 3.

Table 4.

34343



-RS-hkfdhsfjkdhsjkfds

Table 5. table

Table 6.

Table 7.

Table 8.

test topic translate

test 123232

USeR test

some test content

```

#include "VtdCustomSut/VtdCustomSut.h"#include "VtdCustomSut/VtdCustomSut.tpp"#include <
#include "VtdCustomSut/VtdCustomSut.h"
#include "VtdCustomSut/VtdCustomSut.tpp"

#include <cstdlib>
#include <exception>
#include <string>
#include <thread>
#include <chrono>
#include <memory>
#include <iostream>
#include <boost/program_options.hpp>
#include <boost/filesystem.hpp>
#include <boost/asio/io_service.hpp>
#include <boost/asio/posix/stream_descriptor.hpp>
#include <boost/asio/read.hpp>
#include <boost/system/error_code.hpp>

#include <VtdBusApi/BusCommunicator.h>
#include <VtdStateMachine/VtdComponent.tpp>
#include <VtdStateMachine/VtdComponentStateMachine.h>
#include <VtdStateMachine/Util.h>

using namespace boost::asio;

namespace {

/// @brief Read configuration value from program_options or yaml configuration
/// @details program_options has precedence over yaml.
/// Output is unchanged if name isn't found in either program_options or yaml.
template<class T>
void getConfigValue(
    const std::string& name,          ///< Parameter name
    const po::variables_map& vm,      ///< program_options variable map
    const YAML::Node& config,         ///< YAML node for component configuration
    T& output                         ///< [out] Variable to store output if found
)
{
    if (vm.count(name))
    {
        output = vm[name].as<T>();
    }
    else if (config[name] && !config[name].IsNull())

```

```

    {
        output = config[name].as<T>();
    }
    else
    {
        // MISRA/AUTOSAR
    }
}

/// @brief Read configuration value from program_options or yaml configuration
/// @details program_options has precedence over yaml.
/// Output is unchanged if name isn't found in either program_options or yaml.
/// Specialization for output of std::optional<> type.
template<class T>
static void getConfigValue(
    const std::string& name,          ///< Parameter name
    const po::variables_map& vm,      ///< program_options variable map
    const YAML::Node& config,         ///< YAML node for component configuration
    std::optional<T>& output)         ///< [out] Variable to store output if found
{
    if (vm.count(name))
    {
        output = vm[name].as<T>();
    }
    else if (config[name] && !config[name].IsNull())
    {
        output = config[name].as<T>();
    }
    else
    {
        // MISRA/AUTOSAR
    }
}

/// @brief Set output with configuration file or program options
/// @details program_options has precedence over yaml.
template<class T>
void getConfigDump(
    const std::string& name,          ///< Parameter name
    const po::variables_map& vm,      ///< program_options variable map
    const YAML::Node& config,         ///< YAML node for component configuration
    T& output                         ///< [out] Variable to store output if found
)
{
    if (vm.count(name))
    {
        output = vm[name].as<T>();
    }
    else
    {
        output = config.as<T>();
    }
}

```

```

}

int main
(
    int argc,          ///< [in] argument count
    char* argv[]       ///< [in] array of arguments
)
{
    namespace po = boost::program_options;

    std::string name;
    std::string entityNameRef;
    uint32_t sensorId;
    vtdCustomSut::ChannelInfo channelInfo;
    std::map<int, vtdCustomSut::ChannelInfo> sensorInfo;
    po::options_description desc("Usage");

    std::string componentName = "sut";

    desc.add_options()
        ("help", "Show help message.")
        ("name", po::value<std::string>(&name), "Name of the SuT")
        ("entityNameRef", po::value<std::string>(&entityNameRef),
         "Name of the reference entity")
        ("sensorId", "ID of sensor to use.")
        ("channelId", "ID of channel from which the sensor is read.")
        ("type", "Type of sensor.")
        ("channelName", "Name of the channel.");
    Vtd::Logger::addCommonCliOptions(desc);

    po::variables_map vm;
    po::store(po::parse_command_line(argc, argv, desc), vm);
    po::notify(vm);
    // General logger instance unrelated to concrete component instance
    Vtd::Logger loggerGeneral(componentName, &vm);

    auto configList = Vtd::getYamlConfig(componentName);
    if (vm.count("help"))
    {
        std::cout << desc << "\n";
        return EXIT_SUCCESS;
    }
    if (!vm.count("name"))
    {
        loggerGeneral.critical("Provide the SuT name that needs to be loaded. No default");
        return EXIT_FAILURE;
    }
    else if (Vtd::containsDuplicateConfiguration(configList))
    {
        loggerGeneral.critical("Duplicate SuT names found in the configuration file");
        return EXIT_FAILURE;
    }
    else
    {
        bool sutFound = false;

```

```

for (size_t sutIndex = 0; sutIndex < configList.size(); ++sutIndex)
{
    auto configComponent      = configList[sutIndex];
    if (configComponent["name"].as<std::string>() == vm["name"].as<std::string>()
    {
        sutFound = true;
        std::string componentName = Vtd::createComponentInstanceName(componentName, configComponent["name"]);

        // Logger instance related to concrete component instance
        Vtd::Logger logger(componentInstanceName, configComponent, &vm);

        getConfigValue("entityNameRef", vm, configComponent, entityNameRef);
        for (const YAML::Node& input : configComponent["inputs"])
        {
            getConfigDump("sensorId", vm, input["channel"]["sensorIdRef"], sensorInfo);
            getConfigDump("channelId", vm, input["channel"]["id"], channelInfo);
            getConfigDump("channelName", vm, input["channel"]["channelName"], channelInfo);
            getConfigDump("type", vm, input["channel"]["type"], channelInfo);
            sensorInfo[sensorId] = channelInfo;
        }
        try
        {
            boost::asio::io_context io_context;
            boost::asio::executor_work_guard<boost::asio::io_context::executor_type> work_guard(
                boost::asio::make_work_guard(io_context));
            VtdBus::BusCommunicator busCommunicator(logger);
            Vtd::VtdComponentStateMachine stateMachine;

            vtdCustomSut::VtdCustomSut customSut(busCommunicator, stateMachine,
                                                  sensorInfo, entityNameRef, componentName);

            stateMachine.start();
            io_context.run();
        }
        catch (std::exception& e)
        {
            logger.critical("{} ", e.what());
            return EXIT_FAILURE;
        }
        break;
    }
}
if (!sutFound)
{
    loggerGeneral.critical("No SuT configuration present for the name entry provided");
    return EXIT_FAILURE;
}

return EXIT_SUCCESS;
}

vbhghjgjhghjh

```

```

#include "VtdCustomSut/VtdCustomSut.h"
#include "VtdCustomSut/VtdCustomSut.hpp"

#include <cstdlib>
#include <exception>
#include <string>
#include <thread>
#include <chrono>
#include <memory>
#include <iostream>
#include <boost/program_options.hpp>
#include <boost/filesystem.hpp>
#include <boost/asio/io_service.hpp>
#include <boost/asio/posix/stream_descriptor.hpp>
#include <boost/asio/read.hpp>
#include <boost/system/error_code.hpp>

#include <VtdBusApi/BusCommunicator.h>
#include <VtdStateMachine/VtdComponent.hpp>
#include <VtdStateMachine/VtdComponentStateMachine.h>
#include <VtdStateMachine/Util.h>

using namespace boost::asio;

namespace {

/// @brief Read configuration value from program_options or yaml configuration
/// @details program_options has precedence over yaml.
/// Output is unchanged if name isn't found in either program_options or yaml.
template<class T>
void getConfigValue(
    const std::string& name,          ///< Parameter name
    const po::variables_map& vm,      ///< program_options variable map
    const YAML::Node& config,         ///< YAML node for component configuration
    T& output                         ///< [out] Variable to store output if found
)
{
    if (vm.count(name))
    {
        output = vm[name].as<T>();
    }
    else if (config[name] && !config[name].IsNull())
    {
        output = config[name].as<T>();
    }
    else
    {
        // MISRA/AUTOSAR
    }
}

/// @brief Read configuration value from program_options or yaml configuration
/// @details program_options has precedence over yaml.
/// Output is unchanged if name isn't found in either program_options or yaml.

```

```

///      Specialization for output of std::optional<> type.
template<class T>
static void getConfigValue(
    const std::string& name,          ///< Parameter name
    const po::variables_map& vm,      ///< program_options variable map
    const YAML::Node& config,         ///< YAML node for component configuration
    std::optional<T>& output)         ///< [out] Variable to store output if found
{
    if (vm.count(name))
    {
        output = vm[name].as<T>();
    }
    else if (config[name] && !config[name].IsNull())
    {
        output = config[name].as<T>();
    }
    else
    {
        // MISRA/AUTOSAR
    }
}

/// @brief Set output with configuration file or program options
/// @details program_options has precedence over yaml.
template<class T>
void getConfigDump(
    const std::string& name,          ///< Parameter name
    const po::variables_map& vm,      ///< program_options variable map
    const YAML::Node& config,         ///< YAML node for component configuration
    T& output                         ///< [out] Variable to store output if found
)
{
    if (vm.count(name))
    {
        output = vm[name].as<T>();
    }
    else
    {
        output = config.as<T>();
    }
}

int main
(
    int argc,          ///< [in] argument count
    char* argv[]       ///< [in] array of arguments
)
{
    namespace po = boost::program_options;

    std::string name;
    std::string entityNameRef;
    uint32_t sensorId;

```

```

vtdCustomSut::ChannelInfo channelInfo;
std::map<int, vtdCustomSut::ChannelInfo> sensorInfo;
po::options_description desc("Usage");

std::string componentName = "sut";

desc.add_options()
("help", "Show help message.")
("name", po::value<std::string>(&name), "Name of the SuT")
("entityNameRef", po::value<std::string>(&entityNameRef),
 "Name of the reference entity")
("sensorId", "ID of sensor to use.")
("channelId", "ID of channel from which the sensor is read.")
("type", "Type of sensor.")
("channelName", "Name of the channel.");
Vtd::Logger::addCommonCliOptions(desc);

po::variables_map vm;
po::store(po::parse_command_line(argc, argv, desc), vm);
po::notify(vm);
// General logger instance unrelated to concrete component instance
Vtd::Logger loggerGeneral(componentName, &vm);

auto configList      = Vtd::getYamlConfig(componentName);
if (vm.count("help"))
{
    std::cout << desc << "\n";
    return EXIT_SUCCESS;
}
if (!vm.count("name"))
{
    loggerGeneral.critical("Provide the SuT name that needs to be loaded. No default");
    return EXIT_FAILURE;
}
else if (Vtd::containsDuplicateConfiguration(configList))
{
    loggerGeneral.critical("Duplicate SuT names found in the configuration file");
    return EXIT_FAILURE;
}
else
{
    bool sutFound = false;
    for (size_t sutIndex = 0; sutIndex < configList.size(); ++sutIndex)
    {
        auto configComponent      = configList[sutIndex];
        if (configComponent["name"].as<std::string>() == vm["name"].as<std::string>())
        {
            sutFound = true;
            std::string componentNameInstance = Vtd::createComponentInstanceName(configComponent["name"].as<std::string>(),
                                                                                      configComponent["sensorId"].as<int>(),
                                                                                      configComponent["channelId"].as<int>(),
                                                                                      configComponent["channelName"].as<std::string>());

            // Logger instance related to concrete component instance
            Vtd::Logger logger(componentInstanceName, configComponent, &vm);

            getConfigValue("entityNameRef", vm, configComponent, entityNameRef);
        }
    }
}

```

```

    for (const YAML::Node& input : configComponent["inputs"])
    {
        getConfigDump("sensorId", vm, input["channel"]["sensorIdRef"], sensorInfo);
        getConfigDump("channelId", vm, input["channel"]["id"], channelInfo);
        getConfigDump("channelName", vm, input["channel"]["channelName"], channelInfo);
        getConfigDump("type", vm, input["channel"]["type"], channelInfo);
        sensorInfo[sensorId] = channelInfo;
    }
    try
    {
        boost::asio::io_context io_context;
        boost::asio::executor_work_guard<boost::asio::io_context::executor_type>
            boost::asio::make_work_guard(io_context);
        VtdBus::BusCommunicator busCommunicator(logger);
        Vtd::VtdComponentStateMachine stateMachine;

        vtdCustomSut::VtdCustomSut customSut(busCommunicator, stateMachine,
                                              sensorInfo, entityNameRef, component);

        stateMachine.start();
        io_context.run();
    }
    catch (std::exception& e)
    {
        logger.critical("{} ", e.what());
        return EXIT_FAILURE;
    }
    break;
}

if (!sutFound)
{
    loggerGeneral.critical("No SuT configuration present for the name entry provided");
    return EXIT_FAILURE;
}

return EXIT_SUCCESS;
}

```